Estratégia Concursos

Aula 22

Polícia Federal (Agente de Polícia) Passo Estratégico de Informática - Prof. Thiago Cavalcanti

Autor:

Thiago Rodrigues Cavalcanti

18 de Maio de 2023

98276239072 - FABIANO CARBONELL DA SILVA

Noções de programação Python e R. API (application programming interface). Metadados de arquivos

Sumário

Análise Estatística 1
Roteiro de revisão e pontos do assunto que merecem destaque 2
Linguagem Python™ 2
Linguagem R 3.4.2 e RStudio 5.1 7
API (application programming interface) 21
Metadados de Arquivos 23
Aposta estratégica 30
Questões estratégicas 32

Análise Estatística

Assunto	Grau de incidência em concursos similares Cebraspe
4 Conceitos de proteção e segurança. 4.1 Noções de vírus, worms e pragas virtuais. 4.2 Aplicativos para segurança (antivírus, firewall, anti-spyware etc.). 5 Computação na nuvem (cloud computing).	15,07%
3 Redes de computadores. 10 Redes de comunicação. 10.1 Introdução a redes (computação/telecomunicações). 10.2 Camada física, de enlace de dados e subcamada de acesso ao meio. 10.3 Noções básicas de transmissão de dados: tipos de enlace, códigos, modos e meios de transmissão. 11 Redes de computadores: locais, metropolitanas e de longa distância. 11.1 Terminologia e aplicações, topologias, modelos de arquitetura (OSI/ISO e TCP/IP) e protocolos. 11.2 Interconexão de redes, nível de transporte.	13,33%

98276239072 - FABIANO CARBONELL DA SILVA

1 Conceito de internet e intranet. 2 Conceitos e modos de utilização de tecnologias, ferramentas, aplicativos e procedimentos associados a internet/intranet.	9,86%
2.1 Ferramentas e aplicativos comerciais de navegação.	9,86%
2.2 Noções de sistema operacional (ambiente Windows).	9,86%
2.4 Edição de planilhas (ambientes Microsoft Office e LibreOffice).	6,96%
2.1 Ferramentas e aplicativos comerciais de correio eletrônico, de grupos de discussão, de busca, de pesquisa e de redes sociais.	6,09%
2.4 Edição de textos (ambientes Microsoft Office e LibreOffice).	6,09%
9.4 Banco de dados relacionais: conceitos básicos e características. 9.5 Chaves e relacionamentos. 9.6 Noções de mineração de dados: conceituação e características.	5,22%
2.2 Noções de sistema operacional (ambiente Linux).	4,06%
6 Fundamentos da Teoria Geral de Sistemas. 7 Sistemas de informação. 7.1 Fases e etapas de sistema de informação. 8 Teoria da informação. 8.1 Conceitos de informação, dados, representação de dados, de conhecimentos, segurança e inteligência.	3,48%
9 Banco de dados. 9.1 Base de dados, documentação e prototipação. 9.2 Modelagem conceitual: abstração, modelo entidaderelacionamento, análise funcional e administração de dados. 9.3 Dados estruturados e não estruturados.	3,19%
2.3 Acesso à distância a computadores, transferência de informação e arquivos, aplicativos de áudio, vídeo e multimídia.	2,61%
9.7 Noções de aprendizado de máquina. 9.8 Noções de bigdata: conceito, premissas e aplicação.	2,03%
2.4 Edição de apresentações (ambientes Microsoft Office e LibreOffice).	1,45%
12 Noções de programação Python e R. 13 API (application programming interface). 14 Metadados de arquivos.	0,87%

ROTEIRO DE REVISÃO E PONTOS DO ASSUNTO QUE MERECEM DESTAQUE

A ideia desta seção é apresentar um roteiro para que você realize uma revisão completa do assunto e, ao mesmo tempo, destacar aspectos do conteúdo que merecem atenção.

Para revisar e ficar bem preparado no assunto, você precisa, basicamente, seguir os passos a seguir:

Linguagem Python™

As linguagens dinâmicas eram vistas, no passado, apenas como **linguagens script**, utilizadas para automatizar pequenas tarefas. Porém, com o passar do tempo, elas cresceram, amadureceram e conquistaram seu espaço no mercado, a ponto de chamar a atenção dos grandes fornecedores de tecnologia.

98276239072 - FABIANO CARBONELL DA SILVA

Em suma, as linguagens de script são linguagens de programação que suportam scripts. Mas o que são scripts? São programas escritos para um ambiente de execução que pode interpretar (e, não, compilar) e automatizar a execução de tarefas ou instruções que poderiam ser executadas uma de cada vez por um operador humano. Por exemplo, é possível criar um script para desligar o computador automaticamente após um determinado tempo, ou ainda desabilitar e habilitar a placa de rede de forma programada.

Python é uma linguagem de programação de alto nível, interpretada, imperativa, multiparadigma, e de tipagem forte e dinâmica. Vamos entender essas características:

- *Alto nível = linguagem com nível de abstração bastante elevado, bem longe do código de máquina e mais próximo da linguagem humana.*)
- *Interpretada = seu código é executado linha a linha pelo interpretador e, depois, pelo sistema operacional, isto é, a linguagem escrita não é transformada em código de máquina, mas sim interpretada por outro programa.*
- *Multiparadigma = suporta mais de um paradigma de programação. No caso, eles são: imperativo, procedural, funcional e orientado a objetos.*
- *Tipagem forte = o tipo de dado de um valor é do mesmo tipo da variável ao qual este valor será atribuído. Em outras palavras, se uma variável é do tipo inteiro, eu não posso atribuí-la um valor float.*
- *Tipagem dinâmica = verifica-se o tipo de um dado em tempo de execução e, não, em tempo de compilação.*

Python ainda é uma linguagem que suporta a grande maioria das técnicas de programação orientada a objetos, além de realizar tratamento de exceções. Tudo na linguagem Python é um objeto e sua sintaxe não é baseada diretamente em qualquer linguagem usada comumente. Em vez de vetores, ela inclui três tipos de estrutura de dados: listas, listas imutáveis e hashes, que são chamados de dicionários.

Os programas desenvolvidos em Python são automaticamente compilados para o formato portável chamado de bytecode, essa característica faz com que esses programas utilizem a linguagem básica e a biblioteca padrão e assim possam executar da mesma forma no Linux, Windows e outros sistemas operacionais apenas dependendo de um interpretador Python.

Especificações

- Funcionalidades para expressões regulares; sockets; threads; funcionalidade para data/tempo; analisadores XML; analisadores para arquivos de configuração; funcionalidades para manipulação de arquivos e diretórios; persistência de dados; capacidade para unidades de testes; bibliotecas clientes para os protocolos HTTP, FTP, IMAP, SMTP e NNTP.
- Distribuição sob uma licença própria (compatível com a GPL), que impõe poucas restrições. É permitida a distribuição, comercial ou não, tanto da linguagem quanto de aplicações desenvolvidas nela, em formato binário ou código fonte, bastando cumprir a exigência de manter o aviso de Copyright da PSF (Python Software Foundation).

98276239072 - FABIANO CARBONELL DA SILVA

- Permite a definição de funções por meio da palavra-chave **def**, seguida do nome da função e parênteses (Exemplo: **def NomeFuncao(parametrosFuncao)**). A endentação também é muito importante, visto que – diferente de outras linguagens que possuem blocos limitados por chaves (**{ }**) ou palavras-chaves (**begin/end**) – em Python blocos são delimitados pela própria endentação/tabulação.

Em Python, o código é agrupado através da endentação, ou seja, a endentação vai dizer se uma instrução está dentro de um bloco ou de outro.

- Requer uma endentação padronizada. Em outras linguagens, como C/C++ ou JavaScript, a endentação não é necessária devido aos delimitadores de blocos, sendo utilizada somente para melhor visualização. As IDEs que suportam Python têm, em geral, a função de endentação automática, visto que é um pouco complicado ficar endentando tudo na mão.

Operador	Descrição
+	soma
-	subtração
*	multiplicação
/	divisão
//	divisão de inteiros
**	potenciação
%	módulo (resto da divisão)

Operadores Aritméticos

Operador	Descrição
==	igual
!=	diferente
>	maior
<	menor
>=	maior ou igual
<=	menor ou igual
not	negação
and	conjunção

98276239072 - FABIANO CARBONELL DA SILVA

or	disjunção

Operadores Lógicos

- Os tipos primitivos de Python podem ser simples ou compostos. Os Tipos Simples são: int (boolean), long, float e complex. Os Tipos Compostos são list, tuple, string e dictionary. Para leitura de dados, a linguagem pode utilizar a função embutida raw_input( ), que lê uma string do usuário e armazena em uma variável – seria similar a função scanf( ) da linguagem C.
- O tratamento de exceções é realizado por meio do bloco try-except. Vamos ver um pequeno exemplo de divisão por zero:

```
(x,y) = (5,0)
try:
        z = x/y
except ZeroDivisionError:
        print "Divisão por Zero!"
```

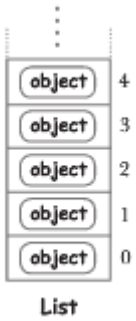
Quando se utiliza métodos em Python, convencionase nomear o primeiro argumento como self (Ex: def metodoQualquer(self, xpto1, xpto2)). Além disso, quando uma classe é instanciada como um objeto, o método construtor (__init__) é opcionalmente invocado. Diferente de outras linguagens, é possível acessar atributos da instância diretamente, sem precisar de métodos.

O interpretador Python executa o código de cima para baixo, uma linha de cada vez. Não é necessário terminar as linhas com ';' em Python e não há a noção de uma função main() como em outras linguagens. Além disso, não é necessário definir os tipos das variáveis e permite a definição de tipos dinâmica.

Listas

98276239072 - FABIANO CARBONELL DA SILVA

Um grande destaque do Python são as listas. As listas são objetos mutáveis e existem diversas formas de trabalhar com listas em Python.



As listas se parecem muito com vetores (arrays), porém elas podem crescer (ou diminuir) de acordo com a necessidade do sistema, trazendo maior flexibilidade.

Por exemplo, vamos criar uma lista que conterá as vogais:

```
vogais = ['a', 'e', 'i', 'o', 'u']
```

Agora vamos supor que queremos saber se alguma palavra contém vogais.

```
palavra = "Magali"
```

Antes de mais nada, perceba que palavra é do tipo String, que no fundo é também uma lista de caracteres. Vejamos:

```
for letra in palavra:
    if letra in vogais:
        print(letra)
```

A saida desse código será: **a a i**

O operador **in** serve para iterar sobre todos os elementos da lista. Ou seja, o código irá verificar para cada letra na palavra “Magali” se ela está na lista de vogais.

Funções

98276239072 - FABIANO CARBONELL DA SILVA

Funções são a pedra fundamental de programas Python.

Um conjunto de funções formam módulos e um conjunto de módulos forma a biblioteca padrão. As funções são identificadas através da palavra reservada. E caso haja algum valor de retorno, a palavra reservada é return. Veja o exemplo abaixo.

```
def nome_descritivo (argumentos_opcionais) :
        """String de
        documentação"""
        # código da função aqui
        return valor_opcional
```

As questões de Python são em sua maioria de análise de código, isso é bom, porém é importante estar familiarizado com a sintaxe.

Linguagem R 3.4.2 e RStudio 5.1

Conceitos básicos

R é uma linguagem computacional que permite que usuário programe algoritmos e utilize ferramentas que foram programadas por outras pessoas. O R é uma **linguagem orientada a objetos** criada em 1996 por Ross Ihaka e Robert Gentleman que aliada a um ambiente integrado permite a **manipulação de dados, realização de cálculos e geração de gráficos**.

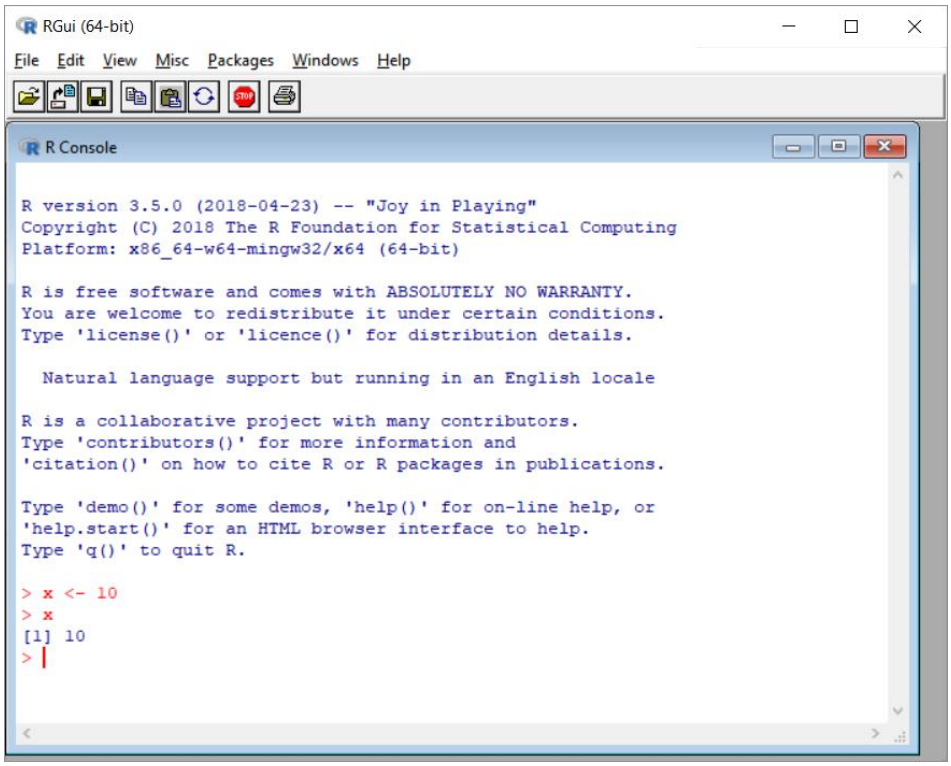
Semelhante à linguagem S desenvolvida pela AT&T's Bell Laboratories e que já é utilizada para análise de dados, mas com a vantagem de **ser de livre distribuição**. Para a instalação do R basta conectar-se ao site http://cran.r-project.org, em CRAN "Comprehensive R Archive Network" escolher o local mais próximo de onde você encontra-se e fazer o download da última versão.

Podemos observar, portanto, que o R é uma linguagem de programação com foco em **análise de dados**. Ou seja, voltada à interação dinâmica com os dados e modelos. Além disso, R é gratuito e de código aberto e seu ambiente permite explorar dados interativamente; mas, à medida que a análise evolui, é uma linguagem de programação completa para desenvolver e automatizar soluções, desenvolver *software* (pacotes).

Além disso, R é uma linguagem **interpretada** e uma ferramenta poderosa para manipulação, processamento, visualização e análise de dados, bem como simulações e modelagem estatísticas.

98276239072 - FABIANO CARBONELL DA SILVA

A instalação padrão do R vem com uma interface gráfica para o usuário (Graphical User Interface – GUI). No Windows, você encontrará o R no menu iniciar e no Mac você verá o ícone do R entre seus aplicativos. No meu caso, consigo encontrar o executável RGui (64-bits) na busca do Windows. Ao executar o mesmo, a seguinte interface é aberta:

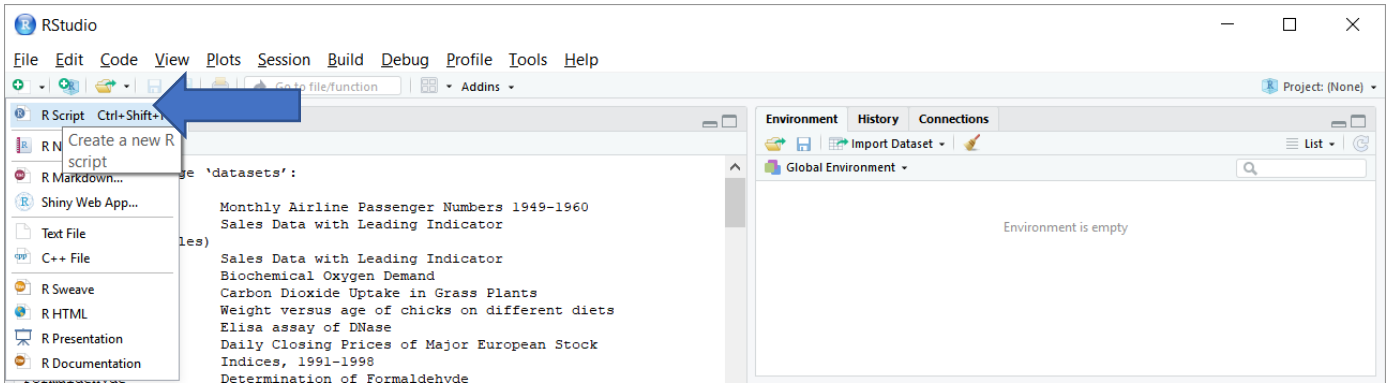


Apesar de o R vir com uma interface gráfica interessante, existe um **Ambiente de Desenvolvimento Integrado** (*Integrated Development Environment- IDE*) chamado **RStudio**, com várias funcionalidades e gratuito. O RStudio possui algumas vantagens em relação ao R Gui:

- Highlight do código;
- Autocomplete;
- Match automático de parênteses e chaves;
- Interface intuitiva para objetos, gráficos e script;
- Criação de "projetos" com interface para controle de versão;
- Facilidade na criação de pacotes;
- Interação com HTML, entre outras.

Você pode abrir o RStudio em seu computador e iniciar um novo Script em "File" -> "New File" -> "New RScript". Você também pode fazer isso com **CTRL + SHIFT + N** ou acessando o botão abaixo.

98276239072 - FABIANO CARBONELL DA SILVA

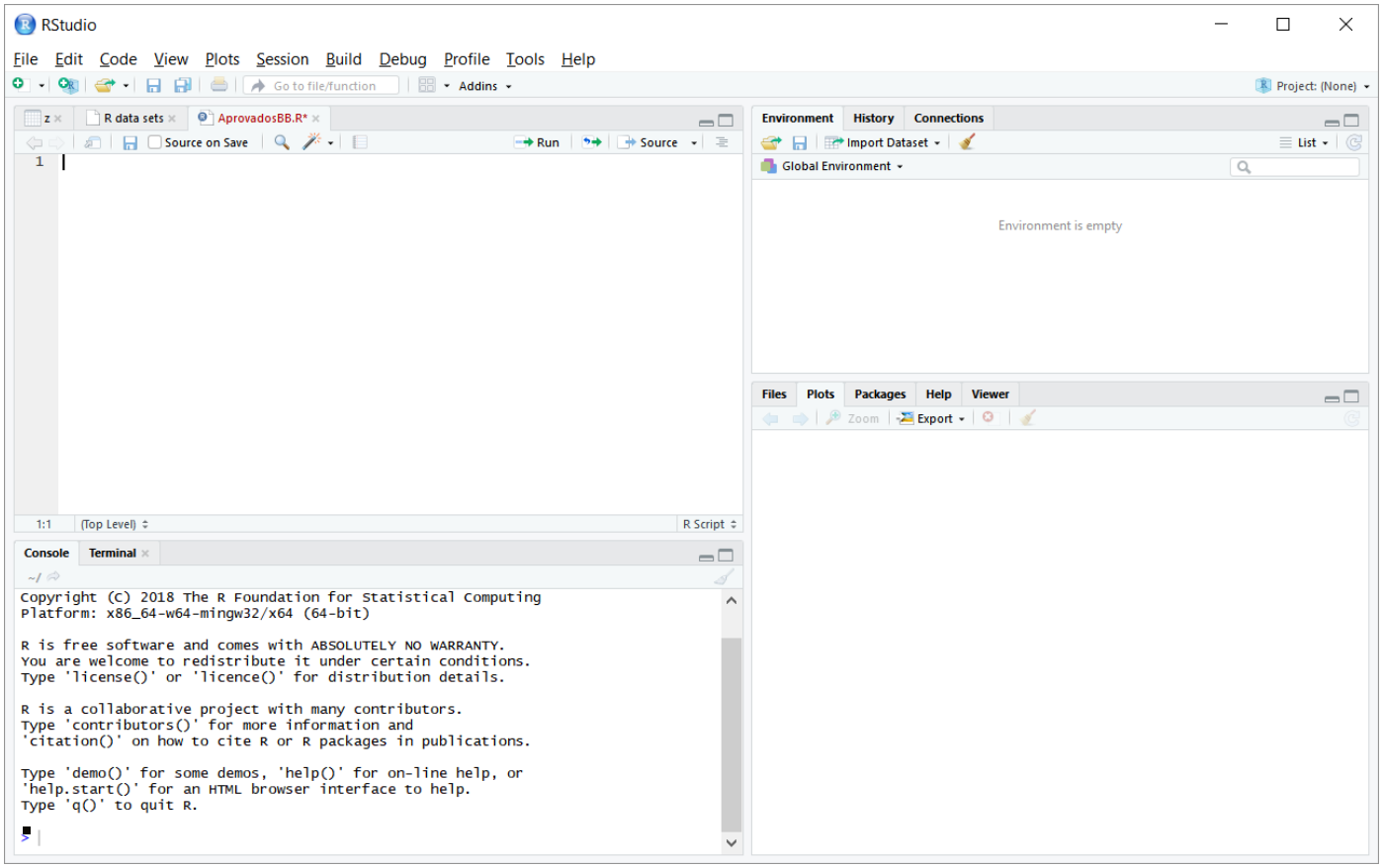


Podemos dividir a tela do RStudio em quatro grandes áreas:

1. **Script**: A tela superior esquerda do RStudio é o editor de texto onde você vai escrever seus Scripts. Ele possui ***code highlighting*** entre outras funcionalidades.
2. **Console**: No canto inferior esquerdo fica o console. O console nada mais é do que uma seção aberta de R, em que os comandos são executados.
3. **Área de trabalho e histórico**: Ficam no canto superior direito. Os objetos criados e o histórico dos comandos podem ser acessados ali.
4. **Arquivos, Gráficos, Pacotes, Ajuda**: Ficam no canto inferior direito. Você pode explorar pastas e arquivos diretamente do RStudio na aba "Files"; os gráficos que forem feitos apareceram na aba "Plots". Os pacotes instalados em sua máquina estão listados em "Packages". As ajudas das funções aparecem em "Help". E o "Viewer" serve para visualização de páginas em HTML e JavaScript.

Essas áreas podem ser vistas na figura a seguir:

98276239072 - FABIANO CARBONELL DA SILVA



TOME NOTA!

Acostume-se a escrever o código no Script ao invés de ficar escrevendo diretamente no console. Para começarmos a nos familiarizarmos com o RStudio, escreva o código abaixo no Script:

> 1 + 1

E aperte **CTRL+ENTER** (**CMD+ENTER** no mac). Isso envia o comando para o console e o resultado é exibido logo abaixo.

> 1+1

[1] 2

Agora escreva o seguinte código no Script.

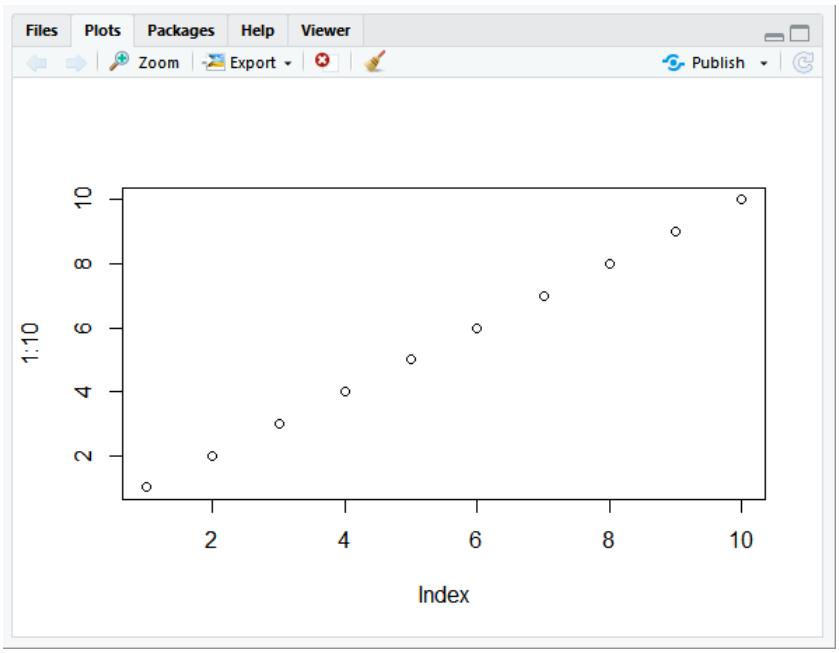
98276239072 - FABIANO CARBONELL DA SILVA

```
> # Gráfico dos números de 1 a 10

> plot(1:10)
```

O primeiro comando **# Gráfico dos números de 1 a 10** é, na verdade, um **comentário**. Comentários nos scripts do R são precedidos do símbolo #, e tudo que estiver após # **não será executado**. *É uma boa prática comentar seu código*! Isso faz com que ele seja de fácil manutenção, tanto para você mesmo (acredite, depois de um tempo você não lembrará o que fez) quanto para seus colegas.

O segundo comando diz ao R para plotar um gráfico. Aperte **CTRL+ENTER** nas duas linhas. O gráfico aparecerá no canto inferior direito do RStudio.



Linguagem R

Agora que já sabemos alguns conceitos básicos sobre R e o RStudio vamos tentar organizar melhor as ideias apresentando alguns outros pontos importantes da linguagem. Vamos fazer um conjunto de dados simples (em linguagem R, um vetor) que consiste nos números 1, 2 e 4 e nomeá-lo como x:

```
> x <- c (1,2,4)
```

O operador de atribuição padrão em R é **<-**. Você também pode usar =, mas isso é desencorajado, pois não funciona em algumas situações especiais. Observe que não há tipos fixos associados a variáveis. Aqui, atribuímos um vetor a x, mas depois podemos atribuir algo de um tipo de dados diferente a ele.

98276239072 - FABIANO CARBONELL DA SILVA

O c significa **concatenar**. Aqui, estamos concatenando os números 1, 2 e 4. Mais precisamente, estamos concatenando três vetores de um elemento que consistem nesses números. Isso ocorre porque qualquer número também é considerado um vetor de um elemento.

Agora também podemos escrever o seguinte:

```
> q <- c (x, x, 8)
```

que define q como um vetor com os valores (1,2,4,1,2,4,8) (sim, incluindo os duplicados).

Agora vamos confirmar que os dados estão realmente em x. Para imprimir o vetor na tela, basta digitar seu nome. Se você digitar qualquer nome de variável (ou qualquer expressão) enquanto estiver no modo interativo, o R imprimirá o valor dessa variável (ou expressão). Os programadores familiarizados com outras linguagens, como o Python, acharão esse recurso familiar. Para o nosso exemplo, insira isto:

```
> x
```

[1] 1 2 4

Sim, com certeza, x consiste nos números 1, 2 e 4. Outro ponto interessante é que você pode acessar elementos individuais de um vetor via "[ ]". Veja como podemos imprimir o terceiro elemento de x:

```
> x [3]
```

[1] 4

Como em outras linguagens, o seletor (aqui, 3) é **chamado de índice**. Aqueles familiarizados com as linguagens da família ALGOL, como C e C++, devem observar que os elementos dos vetores R são indexados a partir de 1, não do 0. Outro ponto é que os subconjuntos podem ser extraídos de vetores. Veja um exemplo:

```
> x <- c (1,2,4)
> x [2:3]
```

[1] 2 4

A expressão x [2: 3] refere-se ao *subvector* de x que consiste do elementos nas posições 2 e 3, que são 2 e 4. Podemos encontrar facilmente a média e o desvio padrão do nosso conjunto de dados, da seguinte forma:

```
> mean(x)
```

[1] 2,333333

```
> sd (x)
```

[1] 1.527525

98276239072 - FABIANO CARBONELL DA SILVA

Isso demonstra novamente que a simples digitação de uma expressão no *prompt* é suficiente para imprimir seu resultado. Na primeira linha, nossa expressão é a função média aritmética, em inglês ***mean(x)***. O valor de retorno da chamada é impresso automaticamente, sem exigir uma chamada para a função ***print( )*** de R.

Se quisermos salvar a média calculada em uma variável em vez de apenas imprimi-la na tela, poderíamos executar este código:

```
> y <- mean(x)
```

Mais uma vez, vamos confirmar que y realmente contém a média de x:

```
> y
```

[1] 2,333333

Como observado anteriormente, usamos # para escrever comentários, assim:

```
> y # imprime
```

[1] 2,333333

Os comentários são especialmente valiosos para documentar o código do programa, mas também são úteis em sessões interativas, uma vez que R registra o histórico de comandos. Se você salvar sua sessão e retomar mais tarde, os comentários podem ajudá-lo a lembrar o que você estava fazendo.

Por fim, vamos fazer algo com um dos conjuntos de dados internos de R (esses são usados para demonstrações). Você pode obter uma lista desses conjuntos de dados digitando o seguinte:

```
> data ()
```

Um dos conjuntos de dados é chamado de Nile e contém dados sobre o fluxo do rio Nilo. Vamos encontrar a média e o desvio padrão desse conjunto de dados:

```
> mean (Nile)
```

[1] 919,35

```
> sd (Nile)
```

[1] 169.2275

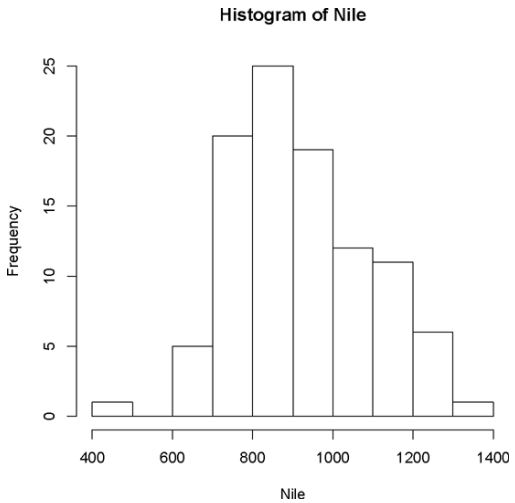
Podemos também traçar um histograma dos dados:

```
> hist (Nilo)
```


98276239072 - FABIANO CARBONELL DA SILVA

Se tivermos usando o RGui, uma janela aparece com o histograma, conforme mostrado na figura abaixo. Este gráfico é simples, mas R tem todos os tipos de gráficos para plotagem. Por exemplo, você pode alterar o número de categorias especificando a variável. Uma chamada ao comando hist(z,breaks=12) desenharia um histograma do conjunto de dados z com 12 caixas ou intervalos.

Você também pode criar rótulos mais agradáveis, fazer uso de cores e fazer muitas outras alterações para criar um gráfico mais informativo e atraente. Quando você se familiarizar mais com o R, poderá construir gráficos de cores ricas e complexas de beleza impressionante, por enquanto seu foco é passar no concurso. Enquanto isso vejamos o gráfico com os dados do Nilo em uma apresentação simples.



Bem, esse é o final de nossa introdução de cinco minutos ao R. Saia de R chamando a função q() (ou alternativamente pressionando ctrl-D no Linux ou cmd-D em um Mac):

```
> q ()
Save workspace image? [y/n/c]: n
```

Esse último *prompt* pergunta se você deseja salvar suas variáveis para que você possa retomar o trabalho mais tarde. Se você responder y, todos esses objetos serão carregados automaticamente na próxima vez que você executar R. Esse é um recurso muito importante, especialmente ao trabalhar com conjuntos de dados grandes ou numerosos. Responder **y** aqui também salva o histórico de comandos da sessão.

A ajuda do R pode seu muito útil quando se deseja saber qual função utilizar ou como utilizar uma função determinada função. Na tabela abaixo são listados alguns comandos para realizar buscas no R:

98276239072 - FABIANO CARBONELL DA SILVA

Ação de Ajuda	Comando
Procurar por "multivariate" em todos os pacotes instalados	`help.search("multivariate")`
Obter ajuda sobre o comando X	`help(comandoX)`
Iniciar ajuda no browser padrão instalado	`help.start()`
Obter ajuda sobre (p.ex.) o pacote cluster	`help(package=cluster)`
Comando que procura objetos (p.ex.) pelo nome anova	`apropos("anova")`
Mostrar exemplos do "comandoX"	`example(comandoX)`
Listar as funções e operações contidas no pacote base do R	`ls("package:base")`

Objetos em R

O que são os Objetos do R? Existem muitos tipos de objetos no R que só passamos a conhecê-los bem com o passar do tempo. Por enquanto vamos aprender os tipos básicos de objetos.

a) **vetores**: uma sequência de valores numéricos ou de caracteres (letras, palavras). Os vetores no R podem ser, entre outros, de tipos: numeric (número comum), integer (inteiro), complex (número complexo), character (texto), logical (lógicos, booleanos).
b) **matrizes**: coleção de vetores em linhas e colunas, todos os vetores dever ser do mesmo tipo (numérico ou de caracteres).
c) **dataframe**: O mesmo que uma matriz, mas aceita vetores de tipos diferentes (numérico e caracteres). Geralmente nós guardamos nossos dados em objetos do tipo data frame, pois sempre temos variáveis numéricas e variáveis categóricas (por exemplo, largura do rio e nome do rio, respectivamente). Similar a uma tabela do SQL, um banco de dados.
d) **listas**: conjunto de vetores, dataframes ou de matrizes. Não precisam ter o mesmo comprimento, é a forma que a maioria das funções retorna os resultados. Muito útil para armazenar resultados de cálculos estatísticos.
e) **funções**: as funções criadas para fazer diversos cálculos também são objetos do R.
f) **factors (fatores)**: fatores são formas de representar objetos categóricos no R.

Todo objeto possui atributos intrínsecos: **tipo** e **tamanho**. Com relação ao tipo ele pode ser: **numérico, caractere, complexo e lógico**. Existem outros tipos, como por exemplo, funções ou expressões, porém esses não representam dados. As funções ***mode***() e ***length***() mostram o tipo e tamanho de um objeto, respectivamente. Por exemplo:

```
x<-c(1,3,5,7,11)
mode(x)
length(x) #mostra o tipo e tamanho do objeto x
```


98276239072 - FABIANO CARBONELL DA SILVA

[1] "numeric"

[1] 5

Observamos acima a atribuição de um vetor de números a variável x. Nomes de variáveis no R podem conter combinações arbitrárias de números, textos, bem como ponto (.) e underscore (_). Entretanto, os nomes não podem começar com números ou underscore.

A tabela abaixo sintetiza os objetos e seus possíveis atributos (tipos). Veja:

Objeto	Tipos	Suporta tipos diferentes
vetor	numérico, caractere, complexo ou lógico	Não
fator	numérico ou caractere	Não
matriz	numérico, caractere, complexo ou lógico	Não
array	numérico, caractere, complexo ou lógico	Sim
data.frame	numérico, caractere, complexo ou lógico	Sim
ts	numérico, caractere, complexo ou lógico	Sim
lista	numérico, caractere, complexo, lógico, função, expressão, etc	Sim

Saber as diferenças entre os diversos tipos de objetos é importante para um uso mais adequado do R. Existem vários tipos de objetos que podem ser criados e manipulados. Já vimos que o R pode trabalhar com vetores - objetos que armazenam mais de um valor. A função c() é usada para criar um vetor a partir de seus argumentos. Os argumentos de c() podem ser escalares ou vetores.

Há ainda outras formas de se gerar um vetor. Por exemplo, para gerar uma sequência de números inteiros usam-se os “dois pontos”. Veja:

a<-1:10 #cria uma sequência de inteiros de 1 a 10

a #exibe o conteúdo do objeto “a”

[1] 1 2 3 4 5 6 7 8 9 10

Uma maneira mais geral de produzir sequências de valores é usando a função seq() que tem como argumentos o início, fim e o incremento da sequência, por exemplo,. seq(1,10,1) é o mesmo que 1:10.

Outra função útil para produzir vetores é a função rep() que retorna o primeiro argumento repetido o número de vezes indicado pelo segundo argumento, o comando rep(1,10) cria um vetor com 10 valores 1.

Constantes

Algumas constantes especiais estão disponíveis.

98276239072 - FABIANO CARBONELL DA SILVA

I **Lógicas**: TRUE, FALSE (evite T e F)

II **Valores especiais**:

Em muitos casos, o resultado de uma operação é infinito ou não determinado. Outras vezes, há valores ausentes em sua base de dados. O R tem objetos especiais para lidar com esses tipos de situação. Por exemplo, quando tiramos o log de um número negativo, obtemos como resultado o valor ***NaN (Not an Number)***. Outros valores de interesse são ***Inf (Infinito)*** e ***NA (Not Available)***. Vejam uma lista de valões especiais abaixo:

	Descrição	Exemplo
– NaN	"not a number"	(0/0) ou log(-1)
– NA	valor faltante	(desconhecido)
– NULL	valor indefinido	(objeto nulo)
– Inf ou -Inf	infinito	(1/0, -1/0)
– pi		3.141593...

O NA representa valores ausentes e é bastante utilizado quando se lida com bases de dados. Muitas funções têm parâmetros que indicam como ela deve lidar com valores ausentes.

III **Outras**:

– LETTERS	"A", "B", ..., "Z"
– letters	"a", "b", ..., "z"
– month.abb	"Jan", "Feb", ..., "Dec"
– month.name	"January", "February", ..., "December"

Funções

Uma das grandes vantagens de usar uma linguagem de programação é automatizar o seu trabalho ou análise. Você será capaz de realizar grande parte do trabalho utilizando as funções internas do R ou de pacotes de terceiros em um *script*. Entretanto, você ganha ainda mais flexibilidade e agilidade criando suas próprias funções. Uma função, no R, é definida da seguinte forma:

98276239072 - FABIANO CARBONELL DA SILVA

```
nomeDaFuncao <- function(arg1, arg2, arg3 = default3,  ...){
  # corpo da função: uma série de comados válidos.
  return(resultado) # opcional
}
```

- o comando **function**() diz para o R que você está definindo uma função.
- os valores dentro dos parênteses de function() são os **argumentos** (ou parâmetros) da função. Argumentos podem ter valores *default*(padrão), que são definidos com o sinal de igualdade (no caso arg3 tem como *default* o valor default3). Existe um parâmetro "coringa" muito útil, o ..., que permite passar argumentos para outras funções.
- dentro das chaves encontra-se o "corpo" da função, isto é, uma série de comandos válidos que serão realizados.
- o comando return() **encerra a função e retorna seu argumento**. O return() é opcional. Caso omitido, a função retorna o último objeto calculado.

Criemos uma função simples que retorna o quadrado de um valor passado como parâmetro:

```
# retorna o quadrado de um número
quadrado <- function(x){
  x^2
}
quadrado(3)
```

```
## [1] 9
```

Funções criam um ambiente local e, em geral, não alteram o objeto ao qual são aplicadas. Isto é, se você passa um valor x para uma função que eleva x ao quadrado, o valor original de x não muda. Funções tomam objetos como argumentos e criam outro objeto, modificado, como resultado. Na maior parte dos casos, a ideia é que uma função no R não tenha efeitos colaterais, isto é, que ela não modifique objetos fora de seu ambiente.

Estruturas de controle

Para escrever funções mais complexas pode ser necessário utilizarmos algumas estruturas de controle como if, ifelse e for. Vejamos algumas informações a respeito deles.

Há ocasiões em queremos ou precisamos executar parte do código **apenas se alguma condição for atendida**. O R fornece três opções básicas para estruturar seu código dessa maneira: if(), if() else e ifelse(). Vejamos cada uma delas.

A estrutura básica do if() é a seguinte:

98276239072 - FABIANO CARBONELL DA SILVA

```
if (condicao) {

  # comandos que
  # serao rodados
  # caso condicao = TRUE

}
```

- O início do código se dá com o comando *if* seguido de parênteses e chaves;
- Dentro dos parênteses temos uma condição lógica, que deverá ter como resultado ou TRUE ou FALSE;
- Dentro das chaves temos o bloco de código que será executado se – e somente se – a condição dos parênteses for TRUE.

A estrutura básica do if() else é a seguinte:

```
if (condicao) {

  # comandos que
  # serao rodados
  # caso condicao = TRUE

} else {

  # comandos que
  # serao rodados
  # caso condicao = FALSE

}
```

- O início do código se dá com o comando if seguido de parênteses e chaves;
- Dentro dos parênteses temos uma condição lógica, que deverá ter como resultado ou TRUE ou FALSE;
- Dentro das chaves do if() temos um bloco de código que será executado se – e somente se – a condição do parênteses for TRUE.
- Logo em seguida temos o else seguido de chaves;
- Dentro das chaves do else temos um bloco de código que será executado se – e somente se – a condição dos parênteses for FALSE.

Os comandos if() e if() else não são vetorizados. Uma alternativa para casos como esses é utilizar a ***função ifelse()***. A função ifelse() tem a seguinte estrutura básica:

```
ifelse(vetor_de_condicoes, valor_se_TRUE, valor_se_FALSE)
```


98276239072 - FABIANO CARBONELL DA SILVA

- o primeiro argumento é um vetor (ou uma expressão que retorna um vetor) com vários TRUE e FALSE;
- o segundo argumento é o valor que será retornado quando o elemento do vetor_de_condicoes for TRUE;
- o terceiro argumento é o valor que será retornado quando o elemento do vetor_de_condicoes for FALSE.

Minha experiência com programação tem demonstrado que usar funções que operam sobre vetores ou matrizes é muito mais performático do que usar outras funções não vetorizadas.

Pacotes

A principal forma de distribuição de códigos no R é por meio de pacotes. Um pacote pode ser entendido como um conjunto de códigos autocontido que adiciona funcionalidades ao R. Para carregar um pacote, use a função library().

Ao carregar um pacote, você está adicionando suas funções ao search da seção, permitindo que você chame estas funções diretamente. Por exemplo, a função mvrnorm, que gera números aleatórios de uma normal multivariada, está no pacote MASS.

```
> Sigma <- matrix(c(10,3,3,2),nrow=2,ncol=2) # Matriz de Var-Covar
> mu <- c(1, 10) # Médias
> x <- mvrnorm(n=100, mu, Sigma) # Tenta gerar 100 obs., mas dá erro!
Error in mvrnorm(n = 100, mu, Sigma) : could not find function "mvrnorm"
```

Para que a função esteja disponível é necessário carregarmos a biblioteca:

```
> library(MASS) # Carrega pacote
> x <- mvrnorm(n=100, mu, Sigma) # Agora funciona
> x
```

Para ver o que está disponível para utilização do R, utilize a função search(). Note que o pacote MASS agora está lá.

```
> search()
[1] ".GlobalEnv"        "package:MASS"      "tools:rstudio"  "package:stats"
[5] "package:graphics" "package:grDevices" "package:utils"  "package:datasets"
```


98276239072 - FABIANO CARBONELL DA SILVA

```
[9] "package:methods"  "Autoloads"        "package:base"
```

Para descarregar um pacote, utilize a função detach().

```
> detach(package:MASS)
```

Às vezes pacotes tem o mesmo nome de funções. Neste caso, se ambos forem carregados, a função que prevalece é a do pacote que foi carregado por último. Uma outra forma de resolver isso é usar o nome do pacote e o operador "::" antes de chamar a função. Neste caso não há ambiguidade.

```
> x <- MASS::mvrnorm(n=100, mu, Sigma)
```

Grande parte dos pacotes do R estão centralizados em um repositório chamado CRAN (The Comprehensive R Archive Network), com diversos espelhos ao redor do mundo.

Com isso terminamos nossa rápida introdução a linguagem R e ao R Studio. Nosso objetivo foi apresentar apenas conceitos básicos sobre o assunto, para você ter uma ideia o curso de R lecionado no Banco Central tem 40 horas aulas ou duas semanas de aulas.

Antes de partir pra o próximo assunto gostaria que você fizesse uma leitura em diagonal neste slide (*http://www.mec.ita.br/~denise/tutorials/IntroR.pdf*). Ele faz uma apresentação de algumas possibilidades gráficas do R, bem como apresenta algumas funções úteis como o summary() e names().

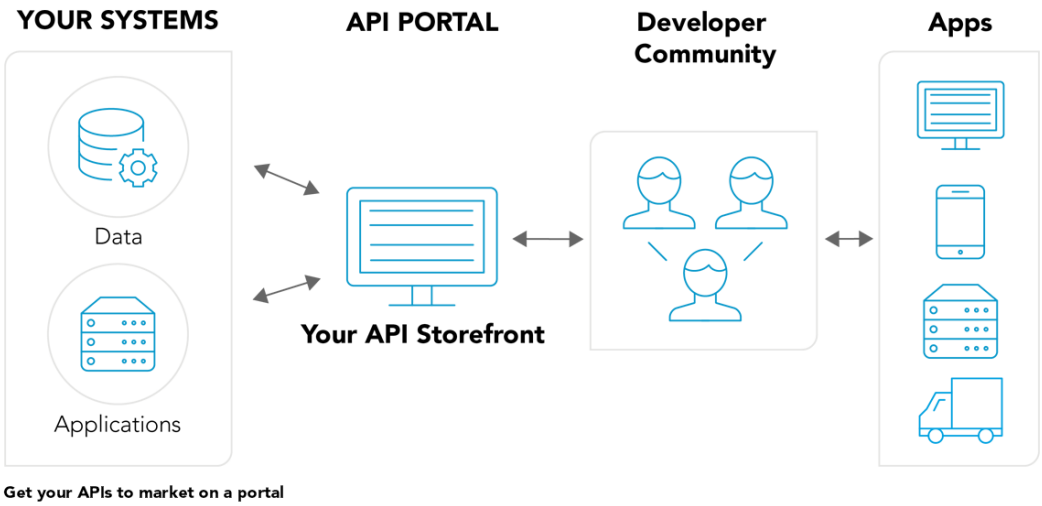
API (application programming interface)

Uma Interface de Programação de Aplicações é um sistema intermediário (conjunto de rotinas e padrões de programação) de software que permite que duas aplicações ou mais "conversem" entre si. Uma API é criada quando uma empresa de software tem a intenção que outros desenvolvedores possam criar produtos associados ao seu software. Por exemplo, o Google Maps por meio do seu código original, permite que muitos outros sites e aplicações utilizem os dados do Google Maps adaptando-o da melhor forma a fim de utilizar esse serviço.

Para entender o funcionamento de uma API, vamos fazer uma analogia com um restaurante. Quando chegamos ao restaurante, chamamos o garçom, que na analogia seria a API, ele traz o cardápio para que possamos fazer o pedido (request) e leva-lo até a cozinha (software ou sistema). A cozinha prepara o pedido e o garçom retorna com o seu prato (response). Ou seja, uma API funciona como um mensageiro, realizando a comunicação entre as aplicações.

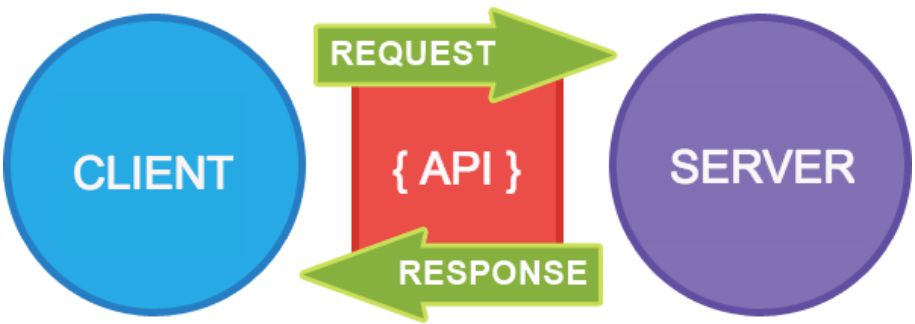
Abaixo temos uma imagem para ajudar a esclarecer o conceito de API.

98276239072 - FABIANO CARBONELL DA SILVA



Conceito de API

Agora vamos apresentar um exemplo de funcionamento de uma API na área de informática. Quando você vai comprar uma passagem aérea, você tem duas opções para encontrar a passagem mais barata. A primeira seria acessar o site de cada uma das companhias inserir data, origem e destino e assim obter os valores disponíveis. A outra opção, é acessar um site que realiza essa busca e retorna com os dados de todas as companhias, por exemplo decolar.com, submarinoviagens.com.br, passagenspromo.com.br (propaganda gratuita! Se algum desses sites quiser me patrocinar, pode entrar em contato comigo através das minhas redes sociais: Facebook® (*profthiagocavalcanti*) ou *Instagram* ☺). *Voltando ao foco da aula. O questionamento que fica: como esses sites têm acesso à base de dados de cada companhia aérea? Eles não têm! Cada companhia oferece sua própria API e dessa forma os sites realizam a pesquisa. Eles informam a API os dados da busca, a API acessa sua base de dados com uma requisição (request), realiza a procura e retorna com a resposta (response).*



98276239072 - FABIANO CARBONELL DA SILVA

A API que foi explicada acima é a mais como e se chama API Web. Entretanto, nós podemos entender API também como uma biblioteca de software, ou seja, uma especificação que descreve e prescreve comportamentos esperados de um software. Dessa forma, podemos dizer que a API é a documentação que determina como um programador pode realizar uma tarefa através de uma biblioteca. A API em seu conceito mais abstrato, é um conjunto de regras para realizar uma tarefa.

Esse assunto NUNCA caiu diretamente em uma prova. Então, sugiro que não se aprofunde muito, o que expliquei até aqui é suficiente para o concurso da Polícia Federal. Qualquer dúvida, pode mandar lá no fórum, que terei o maior prazer em responder.

Metadados de Arquivos

De forma abrangente, os metadados são dados sobre outros dados, ou seja, são marcas ou pontos de referência dos dados, que permitem circunscrever a informação sob todas as formas, criando resumos de informações sobre a forma ou o conteúdo de uma fonte (dado).

A maioria das evidências digitais é armazenada no sistema de arquivos do computador, mas entender como os sistemas de arquivos funcionam é um dos conceitos tecnicamente mais desafiadores para um investigador digital, porque existe pouca documentação sobre esse assunto.

As arquiteturas de dados modernas prometem a capacidade de acesso a diferentes tipos de dados para um número crescente de consumidores dentro de uma organização. Porém, sem uma governança adequada, possibilitada por **uma base sólida de metadados**, essas arquiteturas se mostram apenas uma promessa que, em última análise, não conseguem entregar um resultado consistente.

Vamos fazer uma analogia com a logística distribuição de uma empresa para explicar os metadados e por que é essencial gerenciar os dados no ambiente de negócios atual. Quando você está enviando um pacote para um destino internacional, você quer saber onde na rota o pacote está localizado, e receber informações caso algo de errado aconteça com a entrega. As empresas de logística já mantêm manifestos para acompanhar o movimento de pacotes e a entrega bem-sucedida dos mesmos ao longo do processo de envio.

Os **metadados fornecem esse mesmo tipo de visibilidade no ambiente atual, rico em dados**. Os dados estão entrando e saindo das organizações, assim como trafegam dentro das empresas. Acompanhar as alterações de dados e detectar qualquer processo que cause problemas quando você estiver fazendo a análise de dados será difícil se você não tiver informações sobre os dados e o processo de movimentação dos mesmos. Hoje, até mesmo a alteração de uma única coluna em uma tabela de origem pode afetar centenas de relatórios que usam esses dados, o que torna extremamente importante saber de antemão quais colunas serão afetadas.

Os **metadados** fornecem **informações sobre cada conjunto de dados**, como **tamanho, esquema de banco de dados, formato, horário da última modificação, listas de controle de acesso, uso,** etc. O uso de metadados permite o **gerenciamento de uma plataforma e arquitetura de dados escaláveis**, bem como a governança dos dados. Metadados são comumente armazenados em um catálogo central para fornecer aos usuários informações sobre os conjuntos de dados disponíveis.

98276239072 - FABIANO CARBONELL DA SILVA

Os metadados podem ser classificados em três grupos:

1. Os **metadados técnicos** capturam **a forma e a estrutura de cada conjunto de dados**, como o tamanho e a estrutura do esquema ou tipo de dados (texto, imagens, JSON, Avro etc.). A estrutura do esquema inclui os nomes dos campos, seus tipos de dados, seus comprimentos, se eles podem estar vazios e assim por diante. A estrutura é geralmente fornecida por um banco de dados relacional ou pelo título em uma planilha, mas também pode ser adicionada durante a ingestão e a preparação de dados. Existem alguns metadados técnicos básicos que podem ser obtidos diretamente dos conjuntos de dados (por exemplo: tamanho), mas outros tipos de metadados são derivados.
2. Os **metadados operacionais** capturam **a linhagem, a qualidade, o perfil e a proveniência** (por exemplo, quando os elementos de dados chegaram, onde estão localizados, de onde vieram, qual é a qualidade dos dados, etc.). Também pode conter quantos registros foram rejeitados durante a preparação de dados ou uma execução de tarefa, e o sucesso ou fracasso da própria execução. Os metadados operacionais também identificam com que frequência os dados podem ser atualizados ou atualizados.
3. **Metadados de negócios** capturam **o que os dados significam para o usuário final** a fim de tornar os campos de dados mais fáceis de localizar e entender, por exemplo, nomes de empresas, descrições, tags, qualidade e regras de mascaramento. Eles se vinculam à definição de atributos de negócios para que todos estejam **interpretando consistentemente os mesmos dados por um conjunto de regras e conceitos definido pelos usuários corporativos**. Um glossário de negócios é um local central que fornece uma descrição de negócios para cada elemento de dados por meio do uso de informações de metadados.

Veja que os critérios acima estão descrevendo metadados em um contexto global de uma empresa. Essas informações são essências para tomada de decisões ou auxílio para o tratamento adequado das informações. É diferente de um perito que acompanha uma apreensão de computadores e precisa tomar alguns cuidados para não modificar os arquivos bem como os seus metadados.

- Conectar uma mídia original sem proteção de escrita **pode alterar dados ou metadados** de arquivos e essas alterações podem ser questionadas pelas partes envolvidas. Outra atividade essencial ao fazer a cópia é calcular o *hash* dos dados originais e o da cópia. Esses valores devem coincidir, garantindo-se, com isso, a integridade e a cadeia de custódia das evidências digitais.
- Ao se deparar com máquinas que estejam desligadas, vide regra, não se deve ligá-las. O principal motivo dessa recomendação é a preservação dos dados. Ao se ligar as máquinas, o próprio processo de inicialização do sistema operacional fará alterações em alguns dados, e essas alterações podem ser detectadas examinando-se os metadados de carimbo de tempo dos arquivos. Além disso, acessar arquivos de interesse diretamente nas máquinas, também alterará, no mínimo, os dados de tempo dos arquivos. Desse modo, é preferível fazer imagem dos computadores e proceder as análises sobre as imagens.

98276239072 - FABIANO CARBONELL DA SILVA

Informações de metadados podem ser obtidas de diferentes maneiras. Às vezes, elas são codificadas nos conjuntos de dados, outras vezes podem ser inferidas pela leitura do conteúdo; ou as informações podem ser espalhadas pelos arquivos de log que são gravados pelos processos que acessam esses conjuntos de dados.

DATA LAKE

Data Lake é um termo recente, criado pelo CTO (Chief Technical Officer) do Pentaho, James Dixon, para descrever um componente importante no universo da análise de dados e do Big Data. Em todos os casos, os metadados são um elemento-chave no gerenciamento de um Data Lake. A ideia é ter um único repositório dentro da empresa, para que todos os dados brutos estejam disponíveis a qualquer pessoa que precise fazer análise sobre eles. Comumente utiliza-se o Hadoop para trabalhar com os Data Lakes, mas o conceito é bem mais amplo do que apenas Hadoop.

Os metadados são a base que permite que as seguintes características e recursos de *data lake ou lago de dados* sejam alcançados:

A **visibilidade de dados** é fornecida usando o gerenciamento de metadados para rastrear quais dados estão no lago de dados, junto com a origem, o formato e a linhagem. Isso também pode incluir uma exibição de série temporal, na qual você pode ver quais ações foram atribuídas ou executadas e ver exclusões e inclusões de dados. Isso é muito útil se você quiser fazer uma análise de impacto, o que pode ser necessário quando você faz o gerenciamento de alterações ou cria uma plataforma de dados ágil.

A **confiabilidade dos dados** dá a você a confiança de que **suas análises estão sempre sendo executadas nos dados certos, com a qualidade certa, o que também pode incluir a análise dos metadados**. Uma boa prática é usar uma combinação de abordagens top-down e bottom-up. Na abordagem de cima para baixo, um conjunto de regras definidas por usuários de negócios, administradores de dados ou um centro de excelência é aplicado e essas regras são armazenadas como metadados. Por outro lado, na abordagem de baixo para cima, os consumidores de dados podem qualificar ou modificar os dados ou avaliar os dados em termos de usabilidade, atualização, etc. Os recursos de colaboração em uma plataforma de dados se tornaram uma maneira comum de alavancar a "sabedoria das multidões" para determinar a confiabilidade dos dados para um caso de uso específico.

A **criação de perfil** (*profilling*) de dados permite que **os usuários obtenham informações sobre conjuntos de dados específicos e tenham uma noção do formato e do conteúdo dos dados**. Ele permite que os cientistas de dados e os analistas de negócios tenham uma forma rápida de determinar se desejam usar os dados. O objetivo do perfil de dados é fornecer uma visão para os usuários finais que os ajude a entender o conteúdo do conjunto de dados, o contexto no qual eles podem ser usados na produção e quaisquer anomalias ou problemas que possam exigir correção ou proibir o uso dos dados. Em uma plataforma de dados ágil, o perfil de dados deve ser dimensionado para atender a qualquer volume de dados e estar disponível como um processo automatizado de entrada de dados ou como um processo ad hoc disponível para cientistas de dados, analistas de negócios ou administradores de dados que possam aplicar conhecimentos específicos ao assunto.

98276239072 - FABIANO CARBONELL DA SILVA

Ciclo de vida/idade dos dados: é provável que existam **requisitos de envelhecimento diferentes para os dados em seu lago de dados**, e eles podem ser definidos usando metadados operacionais. Os esquemas de retenção podem se basear em regras globais ou em casos de uso de negócios específicos, mas sempre têm como **objetivo traduzir o valor dos dados em uma política de acesso e armazenamento apropriada**. Isso maximiza o armazenamento disponível e dá prioridade aos dados mais críticos ou de alta utilização. As implementações iniciais de lagos de dados muitas vezes negligenciaram o ciclo de vida dos dados, pois o baixo custo de armazenamento e a natureza distribuída dos dados tornaram essa uma prioridade pouco relevante. À medida que essas implementações amadurecem, as organizações estão percebendo que gerenciar o ciclo de vida dos dados é fundamental para manter um *data lake* eficaz e compatível.

Segurança de dados e privacidade: os metadados **permitem o controle de acesso e o mascaramento de dados** (por exemplo, para informações de identificação pessoal) e garantem a **conformidade com a indústria e outras regulamentações**. Como é possível definir quais conjuntos de dados são confidenciais, você pode proteger os dados, criptografar colunas com informações pessoais ou conceder acesso aos usuários certos com base em metadados. Definir conjuntos de dados com metadados de segurança também simplifica os processos de auditoria e ajuda a expor quaisquer pontos fracos ou vulnerabilidades nas políticas de segurança existentes. A identificação de dados privados ou confidenciais pode ser determinada integrando os metadados do lago de dados com soluções de governança de dados corporativos, examinando os dados após a ingestão para procurar padrões comuns (SSN, códigos do setor etc.) ou utilizando o perfil de dados.

Acesso democratizado a dados úteis: os metadados permitem criar **um sistema para ampliar a acessibilidade do usuário final e o autoatendimento (para aqueles com permissões) visando obter mais valor dos dados**. Com uma extensa estratégia de metadados, você pode fornecer um catálogo robusto para os usuários finais, a partir do qual é possível pesquisar e encontrar dados em qualquer número de facetas ou critérios. Por exemplo, os usuários podem localizar facilmente os dados do cliente de um depósito Teradata que contém dados PII, sem precisar conhecer nomes de tabelas específicos ou o layout do lago de dados.

Linhagem de dados e captura de dados alterados: Nos atuais pipelines de produção de dados, a maioria das empresas se concentra apenas nos metadados dos dados de entrada e saída. No entanto, é comum ter vários processos entre os conjuntos de dados de entrada e de saída, e esses processos nem sempre são gerenciados usando metadados e, portanto, nem sempre capturam alteração de dados ou linhagem. Em qualquer análise de dados ou processo de aprendizado de máquina, os resultados são sempre obtidos da combinação de algoritmos específicos em execução sobre conjuntos de dados específicos, por isso **torna-se extremamente importante ter informações de metadados sobre os processos intermediários, a fim de melhorar a análise ao longo do tempo**.

Os *Data Lakes* devem ser arquitetados adequadamente para aproveitar metadados e integrar-se a ferramentas de metadados existentes, caso contrário, criará um buraco no processo de governança de dados das organizações, pois a forma como os dados são usados, transformados e relacionados fora do data lake podem ser perdidos. Uma arquitetura de metadados incorreta muitas vezes pode impedir que os lagos de dados façam a transição de uma caixa de proteção analítica para uma plataforma de dados corporativos.

98276239072 - FABIANO CARBONELL DA SILVA

Em última análise, a maior parte do tempo gasto na análise de dados está preparando e limpando os dados, e **os metadados ajudam a reduzir o tempo de *insight*[1] fornecendo acesso fácil para descobrir quais dados estão disponíveis e mantendo um mapa de rastreamento de dados completo (linhagem de dados)**.

Os tipos e estrutura de metadados

Os registros de metadados são compostos de três partes: **sintaxe**, **estrutura** e **semântica**. A **sintaxe** é o **padrão de codificação**. Funciona como um contêiner para a estrutura e semântica com um conjunto de regras pelas quais o conteúdo deve ser interpretado. RDF (Resource Description Framework), XML (Linguagem de Marcação Extensível), XHTML (Extensible HyperText Markup Language) e DHTML (Dynamic HyperText Markup Language) são exemplos de padrões de codificação.

Todos podem ser usados para conter os metadados relacionados a objetos, embora o RDF e o XML sejam mais usados nesse contexto. Abaixo, o mesmo registro é representado em dois formatos diferentes. O primeiro exemplo é expresso sem codificação e o segundo exemplo inclui codificação XML.

Exemplo 01

Title: Nutrition information
Creator: Cunliffe, Les
Subject: Nutrition; Food – –Labeling
Description: Image shows a montage of nutrition labels taken from different food packaging.
Publisher:
Date: 2012-04-03
Type: Still Image
Format: image/jpeg
Identifier: 40433994
Source: Fotolia.com
Relation:
Rights: Copyright 2012, Les Cunliffe

Exemplo 02

```
<?xml version="1.0" encoding="UTF-8"?>
```

[1] [1] Descobrir algum padrão ou obter um resultado da análise.

98276239072 - FABIANO CARBONELL DA SILVA

```
<oai_dc:dc
xsi:schemaLocation="http://www.openarchives.org/OAI/2.0/oai_dc/#x2003;http://www.openarchives.org/OAI/2.0/oai_dc.xsd">
 <dc:title>Nutrition information</dc:title>
 <dc:creator>Cunliffe, Les</dc:creator>
 <dc:subject>Nutrition; Food--Labeling</dc:subject>
 <dc:description>Image shows a montage of nutrition labels taken from different food packaging</dc:description>
 <dc:publisher></dc:publisher>
 <dc:date>2012-04-03</dc:date>
 <dc:type>Still Image</dc:type>
 <dc:format>image/jpeg</dc:format>
 <dc:identifier>40433994</dc:identifier>
 <dc:source>Fotolia.com</dc:source>
 <dc:relation></dc:relation>
 <dc:rights>Copyright 2012, Les Cunliffe</dc:rights>
</oai_dc:dc>
```

A **estrutura** se encaixa dentro da sintaxe e é composta por **um esquema de metadados**. Enquanto o padrão de codificação dá ao sistema de computador as instruções sobre como interpretar o registro como um todo, a estrutura dá instruções aos humanos sobre como interpretar as informações dentro de tags individuais (por exemplo, criador, título, assunto, etc.). Você pode pensar em padrões de estrutura como rótulos de rótulos prescritos (mas não no conteúdo das tags). No exemplo 02 acima, o padrão da estrutura (*Dublin Core*) está em negrito.

A parte final de um registro de metadados é **a semântica** ou os padrões de conteúdo. Os padrões de conteúdo definem como as informações dentro das tags devem ser formatadas. As instruções de formatação incluem onde os pontos e vírgulas devem ser colocados, como os títulos devem ser formatados (colocação de ponto e vírgula antes do subtítulo), ordem dos nomes (último, primeiro), etc. O uso de padrões de conteúdo garante a uniformidade das informações contidas nas tags, o que é uma parte essencial da produção de metadados interoperáveis e de alta qualidade em nosso mundo dominado por texto.

Em geral, os metadados são informações ocultas adicionais geradas e incorporadas automaticamente em um arquivo digital. A definição de metadados difere dependendo do contexto em que é usado e da comunidade que se refere a ele. Pode ser considerado como informação compreensível pela máquina ou pode ser referida como registros que descrevem registros digitais.

Na verdade, os metadados podem ser subdivididos em três tipos importantes: **descritivo** (incluindo elementos como autor, título, resumo e palavras-chave), **estrutural** (descrevendo como um objeto é constituído e como os elementos são organizados) e

98276239072 - FABIANO CARBONELL DA SILVA

administrativo (incluindo elementos como data e hora da criação, tipo de dados e outros detalhes técnicos).

Por exemplo, as configurações da câmera (como o marcador da câmera, o modelo da câmera, o tempo de exposição, a velocidade ISO, a distância focal, a velocidade do obturador e assim por diante) são armazenadas em muitos metadados. Objetos multimídia como fotografias e vídeos, Microsoft Office e documentos PDF usam metadados para armazenar o nome do autor, nome do computador, tempo total de edição e data e hora de criação e modificação.

Mídias como arquivos MP3 e MP4 podem conter metadados que descrevem o nome do artista e do álbum, a imagem da capa, informações de codificação e assim por diante. Metadados são muito importantes em um contexto forense. Os metadados nos arquivos JPEG podem conter informações de GPS e esses dados (que descrevem os dados) são armazenados em binários e podem auxiliar em uma investigação. Um celular apreendido com foto que mostra a localização da pessoa no dia de um crime pode ser um álibi ou uma prova.

No mundo da informação eletrônica, os metadados técnicos são uma necessidade. Os metadados técnicos informam como o recurso foi criado, como ele é armazenado e preservado e quais especificações do sistema você precisa usá-lo. Como a tecnologia está evoluindo tão rapidamente, os metadados técnicos são essenciais para garantir o acesso contínuo aos recursos à medida que a tecnologia que os contém envelhece.

Metadados administrativos informam quem ou o que possui e/ou mantém o item. Este tipo de metadados apareceu pela primeira vez no mundo dos arquivos, onde aspectos de informação como a procedência eram importantes para a história e contexto de um recurso. Devido à facilidade de reutilização de informações digitais, ter metadados administrativos para rastrear o ciclo de vida e o uso do recurso é importante.

A fácil transferência e reutilização de conteúdo digital também aumentou a importância dos **metadados de direitos** (*copyright*). Os metadados de direitos descrevem os direitos autorais e os termos de uso do recurso descrito. No mundo da impressão, uma declaração de direitos autorais no verso da página de título geralmente era suficiente para atender a essa necessidade de informação. Mas, no mundo eletrônico, os direitos nem sempre são tão claros e, assim as informações sobre os direitos precisam estar intrinsecamente ligadas aos dados.

Embora possa proteger a propriedade intelectual, os metadados de direitos também podem ajudar no uso de um recurso. Da mesma forma que você usa a propriedade para proteger seu direito sobre um item de dados, também é possível querer que as pessoas o usem e reutilizem suas informações, além de expor sua ideia ou imagem em outros locais.

Se você declarar claramente seus direitos sobre as informações e incluir uma licença no estilo *Creative Commons*[2] que conceda o direito de usar seu trabalho desde que seja devidamente citada, você poderá aumentar as chances de que seu trabalho seja usado.

---

[2] ***Creative Commons*** é uma organização não governamental sem fins lucrativos localizada em Mountain View, na Califórnia, voltada a expandir a quantidade de obras criativas disponíveis, através de suas licenças que permitem a cópia e compartilhamento com menos restrições que o tradicional todos direitos reservados.

98276239072 - FABIANO CARBONELL DA SILVA

Em todos dos contextos, "Os metadados são fundamentais para garantir o entendimento a respeito dos dados e que os recursos sobrevivam e continuem acessíveis no futuro." Assim, encerramos nossa última aula do curso!

APOSTA ESTRATÉGICA

A ideia desta seção é apresentar os pontos do conteúdo que mais possuem chances de serem cobrados em prova, considerando o histórico de questões da banca em provas de nível semelhante à nossa, bem como as inovações no conteúdo, na legislação e nos entendimentos doutrinários e jurisprudenciais[3].

Python é uma linguagem de programação de alto nível, interpretada, imperativa, multiparadigma, e de tipagem forte e dinâmica. Vamos entender essas características:

- *Alto nível = linguagem com nível de abstração bastante elevado, bem longe do código de máquina e mais próximo da linguagem humana.*)
- *Interpretada = seu código é executado linha a linha pelo interpretador e, depois, pelo sistema operacional, isto é, a linguagem escrita não é transformada em código de máquina, mas sim interpretada por outro programa.*
- *Multiparadigma = suporta mais de um paradigma de programação. No caso, eles são: imperativo, procedural, funcional e orientado a objetos.*
- *Tipagem forte = o tipo de dado de um valor é do mesmo tipo da variável ao qual este valor será atribuído. Em outras palavras, se uma variável é do tipo inteiro, eu não posso atribuí-la um valor float.*
- *Tipagem dinâmica = verifica-se o tipo de um dado em tempo de execução e, não, em tempo de compilação.*

Comandos básicos

print()	pula para a próxima linha
input()	aguarda o desenvolvedor digitar algo (valor) com uma sequência de caracteres
zip()	empacota uma combinação sequencial em listas

[3] Vale deixar claro que nem sempre será possível realizar uma aposta estratégica para um determinado assunto, considerando que às vezes não é viável identificar os pontos mais prováveis de serem cobrados a partir de critérios objetivos ou minimamente razoáveis.

98276239072 - FABIANO CARBONELL DA SILVA

enumerate()	enumera um objeto ou retoma a sua contagem
range()	cria uma linha sequencial de números
help()	oferece ajuda em determinadas situações
id()	retoma a identidade de um objeto
dir()	retoma os métodos de um objeto
str()	converte um objeto em texto
int()	converte um objeto em número inteiro
if()	para executar em caso de a condição testada ser verdadeira
else()	para executar em caso de a condição testada ser falsa

Linguagem R 3.4.2 e RStudio 5.1

Objetos em R

O que são os Objetos do R? Existem muitos tipos de objetos no R que só passamos a conhecê-los bem com o passar do tempo. Por enquanto vamos aprender os tipos básicos de objetos.

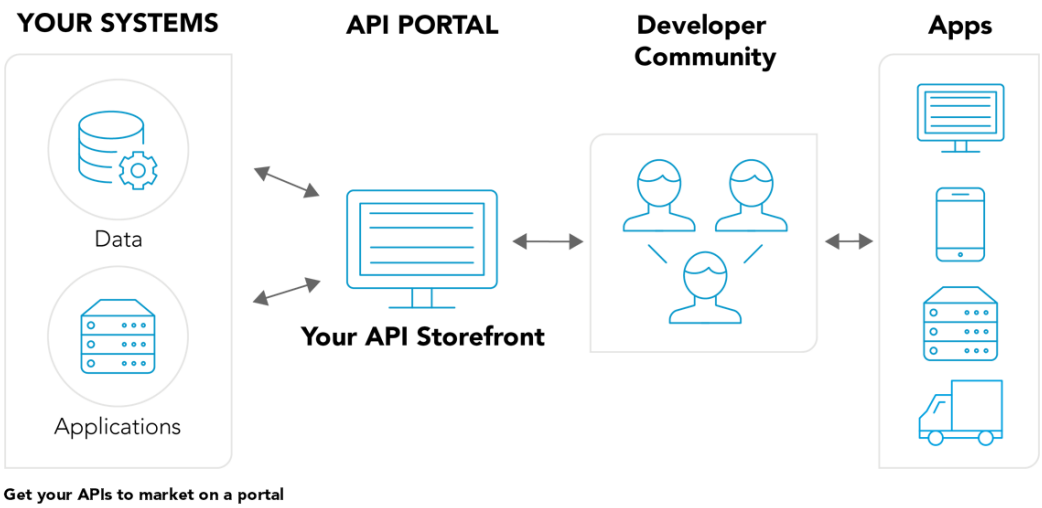
a) **vetores**: uma sequência de valores numéricos ou de caracteres (letras, palavras). Os vetores no R podem ser, entre outros, de tipos: numeric (número comum), integer (inteiro), complex (número complexo), character (texto), logical (lógicos, booleanos).
b) **matrizes**: coleção de vetores em linhas e colunas, todos os vetores dever ser do mesmo tipo (numérico ou de caracteres).
c) **dataframe**: O mesmo que uma matriz, mas aceita vetores de tipos diferentes (numérico e caracteres). Geralmente nós guardamos nossos dados em objetos do tipo data frame, pois sempre temos variáveis numéricas e variáveis categóricas (por exemplo, largura do rio e nome do rio, respectivamente). Similar a uma tabela do SQL, um banco de dados.
d) **listas**: conjunto de vetores, dataframes ou de matrizes. Não precisam ter o mesmo comprimento, é a forma que a maioria das funções retorna os resultados. Muito útil para armazenar resultados de cálculos estatísticos.
e) **funções**: as funções criadas para fazer diversos cálculos também são objetos do R.
f) **factors (fatores)**: fatores são formas de representar objetos categóricos no R.

Todo objeto possui atributos intrínsecos: **tipo** e **tamanho**. Com relação ao tipo ele pode ser: **numérico, caractere, complexo e lógico**. Existem outros tipos, como por exemplo, funções ou expressões, porém esses não representam dados. As funções *mode*() e *length*() mostram o tipo e tamanho de um objeto, respectivamente.

Para entender o funcionamento de uma API, vamos fazer uma analogia com um restaurante. Quando chegamos ao restaurante, chamamos o garçom, que na analogia seria a API, ele traz o cardápio para que possamos fazer

98276239072 - FABIANO CARBONELL DA SILVA

o pedido (request) e leva-lo até a cozinha (software ou sistema). A cozinha prepara o pedido e o garçom retorna com o seu prato (response). Ou seja, uma API funciona como um mensageiro, realizando a comunicação entre as aplicações.



Imprima o capítulo Aposta Estratégica separadamente e dedique um tempo para absolver tudo o que está destacado nessas duas páginas. Caso tenha alguma dúvida, volte ao Roteiro de Revisão e Pontos do Assunto que Merecem Destaque. Se ainda assim restar alguma dúvida, não hesite em me perguntar no fórum.

QUESTÕES ESTRATÉGICAS

Nesta seção, apresentamos e comentamos uma amostra de questões objetivas selecionadas estrategicamente: são questões com nível de dificuldade semelhante ao que você deve esperar para a sua prova e que, em conjunto, abordam os principais pontos do assunto.

A ideia, aqui, não é que você fixe o conteúdo por meio de uma bateria extensa de questões, mas que você faça uma boa revisão global do assunto a partir de, relativamente, poucas questões.

As questões comentadas da aula de hoje serão um pouco diferentes. Como não temos muitas questões do CESPE para alegrar nossa aula e abrilhantar nosso estudo, resolvemos pegar algumas questões de outras

98276239072 - FABIANO CARBONELL DA SILVA

bancas, para dar uma noção de como o assunto é cobrado em concursos. Além disso, resolvemos ressuscitar uma banca histórica de concurso público conhecida como TRC. Criativa, inteligente, dinâmica e objetiva a banca vai focar nos principais pontos do assunto com questões exclusivas para o concurso da Polícia Federal! Faca na caveira!

1. **CEBRASPE (CESPE) - Perito Criminal Federal/Área 1/2018 (e mais 9 concursos)**

Com relação à programação Python e R, julgue o item que se segue.

Considere os seguintes comandos na programação em Python.

```
a = " Hello, World! "
print(a.strip())
```

Esses comandos, quando executados, apresentarão o resultado a seguir.

```
a[0]=Hello,
a[1]=World!
```

Comentários

De acordo com o Docs Python[4], o método strip retorna uma cópia da string removendo os espaços em branco que estiverem no início ou no fim da string. Dessa forma, o código apresentado pelo item retornará "Hello, World!".

Para "quebrar" / "dividir" uma string em partes é utilizado o método split (*docs.python.org/3/library/stdtypes.html#str.split*) que por padrão divide uma string utilizando o espaço em branco como caractere separado. Dessa forma, ao digitar a.split(), irá retornar: 'Hello,', 'Wordl!'.

Gabarito: errado.

---

2. **CEBRASPE (CESPE) - Agente de Polícia Federal/2018**

Julgue o item, relativo a noções de programação Python e R.

Considere o programa a seguir, na linguagem Python.

```
if 5 > 2
{
print("True!")
```

[4] docs.python.org/3/library/stdtypes.html#str.strip

98276239072 - FABIANO CARBONELL DA SILVA

```
}
```

A sintaxe do programa está correta e, quando executado, ele apresentará o seguinte resultado.

```
True!
```

Comentários

Na linguagem Python, os blocos são delimitados por meio da indentação do código. Diferente de outras linguagens tais como C e PHP onde os blocos são delimitados de forma explícita por meio dos caracteres { } (chaves) . Assim, o código apresentado pelo item possui um erro de sintaxe (a utilização das chaves). O código deveria ser escrito da seguinte forma:

```
if 5 > 2:

    print("True!")
```

Gabarito: errado.

---

3. CEBRASPE (CESPE) - Agente de Polícia Federal/2018

Julgue o item, relativo a noções de programação Python e R.

Considere o programa a seguir, na linguagem Python.

```
letras == ["P", "F"]
for x in letras
  {
    print(x)
  }
```

A sintaxe do programa está correta e, quando executado, ele apresentará o seguinte resultado.

```
PF
```

Comentários

Conforme comentei na questão anterior, na linguagem Python, os blocos são delimitados por meio da indentação do código.

Além disso, na primeira linha temos letras == ["P", "F"], é feita uma comparação. Isto é, tenta-se verificar se a variável "letras" é igual a lista ["P", "F"]. Essa primeira linha também gera um erro pois a variável letras ainda nem foi definida.

98276239072 - FABIANO CARBONELL DA SILVA

Analisando o código como um todo, acredito que o objetivo do programa era atribuir uma lista a variável "letras", percorrer essa lista e, imprimir cada item.

O código deveria ser escrito da seguinte forma:

```
letras = [“P”, “F”]

for x in letras:

     print(x, end="")
```

Gabarito: errado.

---

4. (FGV / CM-Caruaru – 2015)

Analise o código Python a seguir.

```
L1=[10,20,30]
L2=[40,50]
L1.append(L2)
print L1
```

Assinale a opção que descreve corretamente o que acontece quando esse programa é executado no Python 2.7.

a) Produz uma mensagem de erro, porque tenta executar uma operação inválida.

b) Exibe “[10, 20, 30, [40, 50]]”.

c) Exibe “[10, 20, 30, 40, 50]”.

d) Exibe “[10, 20, 30], [40, 50]”.

e) Exibe “[]”.

Comentários

O método `lista.`**`append`**`(elemento)` adiciona o elemento no final da lista. No caso, o elemento é uma lista, então a lista será adicionada como elemento. Portanto:

```
>>> L1=[10,20,30]

>>> L2=[40,50]

>>> L1.append(L2)

>>> print L1
```


98276239072 - FABIANO CARBONELL DA SILVA

```
[10, 20, 30, [40, 50]]
```

O gabarito, portanto, é a letra B.

Na versão atual do Python (3x) é OBRIGATÓRIO o uso dos parênteses no comando print. Na versão 3x, o resultado será "SyntaxError: Missing parentheses in call to 'print'. Did you mean print(L1)?"

Gabarito: alternativa B.

---

5. (IESES / IFC-SC – 2015)

Analise o seguinte trecho de código em Python 3.3.2 e assinale a alternativa correta:

```
class Cachorro:
  def tipo(self):
    return "10"

class Basset(Cachorro):
  def tipo(self):
    return "100"

cachorro1 = Cachorro()
cachorro2 = Basset()

print(cachorro1.tipo())
print(cachorro2.tipo())
```

a) Ocorrerá um erro de execução na linha "def tipo(self):" logo abaixo de "class Basset(Cachorro):".

b) A execução de "print(cachorro2.tipo())" mostrará o valor "10" na saída padrão.

c) A execução de "print(cachorro2.tipo())" mostrará o valor "100" na saída padrão.

d) Ocorrerá um erro de execução na linha "print(cachorro2.tipo())".

Comentários

Questão sobre herança, cachorro2 herda da classe Cachorro, porém o atributo 'tipo' é sobrescrito. Passando a retornar "100".

Gabarito: alternativa C.

98276239072 - FABIANO CARBONELL DA SILVA

6. (IESES / IFC-SC – 2015)

Analise as seguintes afirmativas sobre operadores do Python 3.3.2 e assinale a alternativa correta:

I. b %= a é equivalente a b = b % a.

II. <> é o operador de comparação que significa "diferente de" em Python. O operador != também é, mas este é um operador obsoleto e não se recomenda mais sua utilização.

III. O operador ~ é responsável por fazer o "XOR binário", ou seja, realiza um "OU exclusivo" bit a bit.

a) Somente a afirmativa I é verdadeira.

b) As afirmativas I, II e III são verdadeiras.

c) Somente as afirmativas I e III são verdadeiras.

d) Somente as afirmativas II e III são verdadeiras.

Comentários

(I) b %= a é a forma reduzida de representar b = b % a. Assim como b += a equivale é b = b+a; (II) O operador de "diferente de" do Python é o "!="; (III) O "~" é o NOT binário. O XOR binário é o "^".

Gabarito: alternativa A.

---

7. (IESES / IFC-SC – 2015)

Sobre listas em Python 3.1.5:

a) `list.remove(a)` remove o primeiro item da lista cujo valor é a.

b) `list.pop(a)` adiciona um item de valor a ao início da lista.

c) `list.append(a)` adiciona um item à lista cujo índice será a.

d) `list.index(a)` retorna o valor do item cujo índice é a.

Comentários

(a) correto. Caso não exista o valor "a", retorna erro;

(b) O pop na verdade remove o item na posição "a" da lista, ou caso seja chamado como pop() - sem argumento - remove o último item da lista. O que adiciona um item ao topo da lista é o list.insert(0,a), onde 0 pode ser substituído por qualquer posição da lista;

(c) O append adiciona o valor "a" na última posição da lista;

98276239072 - FABIANO CARBONELL DA SILVA

(d) O index retorna à primeira posição que o valor "a" é encontrado na lista, caso não seja encontrado, retorna erro.

Aproveitando que estamos falando de lista, seguem os demais comandos:

- list.extend(L) - é o mesmo que o append, mas ao invés de receber apenas um valor, recebe uma nova lista, que será adicionada ao final de "list";

- list.count(a) - retorna quantas vezes o valor "a" aparece na lista;

- list.sort() - ordena os valores da lista;

- list.reverse() - reverte os elementos da lista (o último vira o primeiro, e assim sucessivamente).

Gabarito: alternativa A.

---

8. (IESES / IFC-SC – 2015)

O conjunto correto de palavras reservadas para a construção de uma estrutura de controle em Python 3.4.3 é:

a) “if”, “elsif” e “else”.

b) “if”, “else if” e “else”.

c) Somente “if”; o restante da estrutura de controle (“senão se” e “senão”) é realizado simplesmente com indentação.

d) “if”, “elif” e “else”.

Comentários

Além do if e do else, o Python também possui o "elif" que é o "senão se".

Gabarito: alternativa D.

---

9. (UERJ / UERJ)

Considere o trecho do programa Python abaixo:

```
def dobra (y):
x = y + y
return x
x = 5
```


98276239072 - FABIANO CARBONELL DA SILVA

```
dobra(x)
dobra(x)
print x
```

O valor impresso ao executarmos o programa é:

a) 5

b) 10

c) 15

d) 25

Comentários

Dentro do método "dobra(y)" a variável x é de outro escopo. Portanto, a variável x que recebe o valor 5, não é alterada em nenhum momento no corpo principal do programa. Seu valor continua sendo 5.

Gabarito: alternativa A.

---

10. (UERJ / UERJ)

A linguagem Python possui a seguinte característica:

a) é uma linguagem compilada

b) exige declaração de código

c) a tupla é um tipo mutável

d) é orientada a objetos

Comentários

(a) É uma linguagem interpretada;

(b) Entendo esse item como declaração de tipo, então não exige, python é uma linguagem dinâmica;

(c) A tupla é equivalente a uma lista, porém é um tipo imutável, seu valor não pode ser modificado;

(d) Correto!

Gabarito: alternativa D.

---

11. (FCC / TRT-MG)

98276239072 - FABIANO CARBONELL DA SILVA

Considere o código fonte Python abaixo.

```
def calcular(n): resultado = [ ] a, b = 0, 1 while a < n: ...I...
return resultado res = calcular(100) print res
```

Para que seja exibido [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89] a lacuna "I" precisa ser preenchida corretamente com:

a) resultado.add(a) a, b = b, a+b

b) resultado.insert(a) a, b = b, a+b

c) resultado.append(a) a, b = b, a+b

d) resultado.add(a) a, b = a, a+b

e) resultado.append(a) a, b = a+b, b

Comentários

Primeiro temos que saber que o correto a ser utilizado é o append, pois queremos inserir valores a cada iteração sem perder os anteriores. Então ficamos entre a letra C e a E. Veja que esse algoritimo utiliza a atribuição de tupla, ou seja, "a, b = 0, 1" equivale a a = 0 e b = 1. Perceba que o algoritmo continua como a sequência de fibonacci, a recebe sempre a soma do item atual e do próximo.

Gabarito: alternativa C.

---

12. (FGV / TJ-BA – 2015)

Analise o trecho de programa, escrito em Pyhton na versão 2.7, mostrado a seguir.

```
X=[]
for i in range(10,1,-1):
X.append(i)
print X[3:]
```

O trecho exibe:

a) [7, 6, 5]

b) [7]

c) []

d) [7, 6, 5, 4, 3, 2]

e) [7, 6, 5, 4, 3, 2, 1]

Comentários

98276239072 - FABIANO CARBONELL DA SILVA

A função range retorna uma lista que inicia no valor 10 e vai até que i seja >1. Com o passo sendo de -1, ou seja, X = [10, 9, 8, 7, 6, 5, 4, 3, 2]. A notação X[3:] significa que é para retornar os elementos da lista começando da posição 3 indo até o final.

Lembre-se que o primeiro índice é "0", então o elemento é "7". Logo será exibido "[7, 6, 5, 4, 3, 2].

Gabarito: alternativa D.

---

13. (FCC / CNMP – 2015)

Considere os fragmentos de programas Phyton a seguir:

Fragmento 1:

```
for n in range(2, 10):
for x in range(2, n):
if n % x == 0:
print n, '=', x, '*', n/x
break
else:
print n, 'é um número primo'
```

Fragmento 2:

```
a = ['Casa', 'Mala', 'Prova']
for x in a:
print x, len(x)
```

É correto afirmar que:

a) o Fragmento 1 está incorreto, pois laços não podem ter uma cláusula else.

b) no Fragmento 2, a instrução for está incorreta, pois ela não pode iterar sobre a.

c) o Fragmento 1 está incorreto, pois não é possível iterar sobre sequências numéricas utilizando a função range.

d) no Fragmento 1 é verificado se o quociente da divisão de n por x corresponde a 0.

e) os dois fragmentos de código estão corretos.

Comentários

Python possui diversos recursos para trabalhar com listas, o fragmento 1 utiliza o range que já vimos em outras questões. Esse fragmento verifica se o resto (não o quociente) da divisão é igual a zero. Já o fragmento 2 utiliza

98276239072 - FABIANO CARBONELL DA SILVA

o operador "in" o comportamento dele é parecido com o for each, ou seja, ele itera sobre todos os objetos do vetor.

Gabarito: alternativa E.

---

14. (FGV / TJ-BA – 2015)

Analise o trecho de programa Python, na versão 2.7, apresentado a seguir.

```
L=[1,2,3,4,5,6,7,8]
print L[::-1]
```

Ao ser executado, o resultado exibido é:

a) [1, 2, 3, 4, 5, 6, 7, 8]

b) [8]

c) [ ]

d) [8, 7, 6, 5, 4, 3, 2, 1]

e) [1]

Comentários

Para exibir a lista podemos definir 3 coisas: a posição inicial, a posição final e se será na ordem reversa ou não. Nesse caso, as posições inicial e final foram deixadas em branco, ou seja, será o valor padrão, que é a lista em toda a extensão. O "-1" indica que a lista será exibida em ordem reversa, ou seja "[8,7,6,5,4,3,2,1]".

Gabarito: alternativa D.

---

15. (TRC / PF-Aquecimento)

Julgue o item abaixo sobre metadados de arquivos em uma análise de disco rígido ou outra mídia de armazenamento:

Ao se deparar com máquinas que estejam desligadas, vide regra, não se deve ligá-las. O principal motivo dessa recomendação é a preservação dos dados. Ao se ligar as máquinas, o próprio processo de inicialização do sistema operacional fará alterações em alguns dados, e essas alterações podem ser detectadas examinando-se os metadados de carimbo de tempo dos arquivos. Além disso, acessar arquivos de interesse diretamente nas máquinas, também alterará, no mínimo, os dados de tempo dos arquivos. Desse modo, é preferível fazer imagem dos computadores e proceder as análises sobre as imagens

Comentários

98276239072 - FABIANO CARBONELL DA SILVA

Essa é uma questão interessante e aborda o assunto da perspectiva investigativa policial. Neste exemplo podemos imaginar que os agentes chegam a uma residência para busca e apreensão e encontram computadores desligados. Para evitar que os dados e os metadados sejam modificados durante a inicialização do computador deve-se levar o computador ainda desligado para a equipe de perícia. Sendo assim, o texto acima encontra-se correto.

Gabarito: certo.

---

16. (TRC / PF-Aquecimento)

Julgue o item abaixo sobre metadados de arquivos em uma análise de disco rígido ou outra mídia de armazenamento:

Conectar a mídia original sem proteção de escrita pode alterar dados ou metadados de arquivos e essas alterações podem ser questionadas pelas partes envolvidas. Outra atividade essencial ao fazer a cópia é calcular o hash dos dados originais e o da cópia. Esses valores devem coincidir, garantindo-se, com isso, a integridade e a cadeia de custódia das evidências digitais.

Comentários

Essa questão mistura aspectos de segurança com a necessidade de mantermos os dados inalterados. Mais uma vez, no âmbito da ação policial investigativa, é necessário garantir a irrefutabilidade das descobertas feitas em arquivos presentes em HD apreendidos. Sem assim, podemos usar a função hash exatamente como descrito no texto.

Gabarito: certo.

---

17. (TRC / PF-Aquecimento)

Essa questão é um exercício prático para ajudar a assimilar o conhecimento sobre a Linguagem R. Nossa sugestão é que você tente usar o R para fazer alguns exercícios que vão ajudar você na fixação do conteúdo. Para tanto, faça o download do RStudio Desktop[5].

Abra o RStudio e tente fazer o seguinte exercício.

1. Remova todos os objetos do ambiente de trabalho.
2. Crie objetos com nomes dados_Agente, dados_Perito, dados_Papi, dados_Escrivao. Faça com que os objetos sejam do tipo numérico, character, lógico e inteiro.
3. Verifique a classe e a estrutura dos objetos criados.
4. Use ls() para listar apenas os dados de Perito. E depois para listar apenas os dados de Agente.

[5] Disponível em: *https://www.rstudio.com/products/rstudio/download/*

98276239072 - FABIANO CARBONELL DA SILVA

5. Use rm() para remover apenas os dados de Escrivao (dica: você vai precisar de usar o resultado de ls()).
6. Salve sua área de trabalho com o nome exercicio_objetos_PF.RData.

Uma possível solução para os itens acima:

1. **rm**(list=**ls**()) *# remove todos os objetos*
2. dados_Agente <- 10.5
   dados_Perito <- "texto"
   dados_Papi <- TRUE
   dados_Escrivao <- 24L
3. **class**(dados_Agente)
   class(dados_Perito)
   class(dados_Papi)
   class(dados_Escrivao)
   str(dados_Agene)
   str(dados_Parito)
   str(dados_Papi)
   str(dados_Escrivao)
4. perito <- **ls**(pattern="Perito")
   agente <- **ls**(pattern="Agente")
5. **rm**(list=agente)
6. **save.image**(file=" exercicio_objetos_PF.RData")

...

Forte abraço e bons estudos.

"Hoje, o 'Eu não sei', se tornou o 'Eu ainda não sei'"

(Bill Gates)

Thiago Cavalcanti

98276239072 - FABIANO CARBONELL DA SILVA

Face: www.facebook.com/profthiagocavalcanti
Insta: www.instagram.com/prof.thiago.cavalcanti
YouTube: youtube.com/profthiagocavalcanti

98276239072 - FABIANO CARBONELL DA SILVA

ESSA LEI TODO MUNDO CONHECE:
PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.

1. Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.

2. Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).

3. Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).

4. Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).

5. Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).

6. Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).

7. Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.

8. O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.

Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.